

Howard (Hao) Ye

hyedailyuse@gmail.com — (865) 456-1194 — Shrewsbury, MA

[LinkedIn](#) — [GitHub](#) — [Portfolio](#)

SUMMARY

DevOps / Platform Engineer with 9+ years of technical depth spanning parallel and distributed storage systems research, cloud infrastructure automation, and containerized application delivery. Hands-on expertise with Ansible, Terraform, Docker, Kubernetes, GitHub Actions, Prometheus/Grafana, WireGuard, and Nginx on GCP/AWS Linux environments. Proven ability to design, automate, and operate production-grade infrastructure independently.

TECHNICAL SKILLS

Infrastructure as Code: Ansible (8-role modular projects, idempotent, Molecule-tested), Terraform (GCP provisioning)

Containers & Orchestration: Docker, Docker Compose, Kubernetes (Deployment, StatefulSet, PVC, ConfigMap, Service)

CI/CD: GitHub Actions, Jenkins (multi-stage pipelines), ansible-lint, yamllint gate checks

Cloud: GCP (Compute Engine, Ubuntu VMs), AWS (EC2, RDS, S3, VPC, Security Groups)

Observability: Prometheus, Grafana, Alertmanager, Node Exporter; custom alert rules (CPU, memory, downtime)

Linux & Networking: Ubuntu/Debian admin, UFW/iptables, fail2ban, SSH hardening, systemd, WireGuard VPN, Nginx, Certbot/TLS

Storage & Performance: Lustre parallel file system, IOR benchmarking, HPC I/O workload analysis, queuing theory (M/M/1)

Languages: Bash, Python, TypeScript, JavaScript, Node.js, C/C++, Go

Databases: PostgreSQL, Redis, SQLite

EDUCATION

MS Computer Engineering (Research Track, formerly PhD candidate) — University of Tennessee Knoxville 2016–2025

EXPERIENCE

Graduate Research Engineer — **University of Tennessee Knoxville** **2017–2025**

Research: Lustre File System Optimization for HPC Cluster Computing

- Benchmarked UTK's ACF production Lustre cluster using IOR to characterize parallel I/O performance across workload types (sequential read/write, random read/write, strided, shared-file, file-per-process), sweeping block sizes from 7 KiB to 256 MiB, up to 24 OSTs with multiple compute nodes scaled in parallel; measured peak read throughput of **~19,000 MiB/s** and up to 27,000+ IOPS.
- Collected and analyzed real Lustre server-side telemetry via OSS `obdfilter` jobstats and MDS jobstats — recording per-job `read_bytes`, `write_bytes`, `getattr`, `open/close` counts across 4 OSSes and the MDS; parsed workload logs covering individual job I/O volumes up to **4.2 GB per snapshot**.
- Built an analytical M/M/1 queueing model and a Colored Petri Net (CPN) simulation framework in OM-NeT++ to model Lustre I/O performance under parallel workloads (fat-tree topology: 64 compute nodes, 200 OSTs, 10 OSSes); calibrated service-rate parameters (OST read ~ 700 MB/s, write ~ 400 MB/s) from IOR measurements; validated model against simulator output for the ACF topology.
- Extended the Markov chain / stochastic modeling toolkit to a federated learning (FL) use case: designed and measured a 4-client smart-home FL system (5 training rounds), modeled round timing with Markov chains to evaluate whether FL-based energy optimization can be achieved without violating per-household comfort constraints.
- Evaluated 10 HPC I/O benchmarks (IOR, IOzone, PostMark, FLASH-IO, HPIO, etc.) as representative application I/O profiles (checkpointing, scientific simulation, metadata-heavy workloads) to identify Lustre tuning levers (stripe count, stripe size, OST placement); surveyed 50+ papers on PFS scheduling, burst buffers, and I/O variability reduction.

Volunteer DevOps & Platform Engineer — MatchingDonors.com (Remote) Jan 2026–Present

- Designed and maintained dual CI/CD pipelines: GitHub Actions (automated lint + test gate on every push) and multi-stage Jenkins pipeline (checkout → install → test → deploy) for controlled production releases.
- Containerized all services with Docker and Docker Compose; authored Kubernetes manifests from scratch — 5-replica Deployment, PostgreSQL StatefulSet with PVC, ClusterIP Service — for scaled agent orchestration on Minikube.
- Wrote shell API smoke test suite (`test-backend-apis.sh`) covering 9+ service modules for automated post-deployment production validation.

PROJECTS

Linux Infrastructure Automation, Hardening & Observability — [\[Repo\]](#)

Stack: Ansible, Terraform, GCP, Docker Compose, Prometheus, Grafana, Alertmanager, Node Exporter, WireGuard, Nginx, Certbot, UFW, fail2ban, GitHub Actions, Molecule

- Engineered 8-role modular Ansible project for full GCP Ubuntu server lifecycle: **base** (system hardening, apt updates), **ssh** (key-only auth, strict `.ssh` permissions), **nginx** (reverse proxy, Jinja2 templates, handlers), **certbot** (Let's Encrypt TLS, 90-day auto-renewal), **monitoring** (Prometheus + Grafana + Alertmanager + Node Exporter), **wireguard** (VPN, iptables POSTROUTING MASQUERADE), **blue-green** (zero-downtime traffic switching), **app** (application deployment).
- Deployed self-hosted observability stack via Docker Compose with 3 custom Prometheus alert rules (CPU > 80%, memory > 85%, service downtime) and Grafana dashboards for real-time system health visibility.
- Enforced IaC quality via GitHub Actions CI (ansible-lint + yamllint on every push); validated role idempotency with Molecule integration tests. Terraform manages GCP compute provisioning upstream.

Kubernetes Deployment — AI Agent Swarm — [\[Repo\]](#)

Stack: Kubernetes, Docker, Node.js/TypeScript, PostgreSQL, Minikube

- Authored production-style K8s manifests from scratch: **agent-swarm.yaml** (5-replica Deployment with env injection), **postgres.yaml** (StatefulSet + PVC for data durability), **agent-service.yaml** (ClusterIP Service).
- Containerized TypeScript AI content pipeline (crawl → Gemini processing → dedup → Bluesky publish) with multi-stage Dockerfile; Docker Compose for local dev, K8s for scaled production.

Cloud-Native Microservices on AWS — [\[Repo\]](#)

Stack: Docker Compose, Node.js/TypeScript, PostgreSQL, Redis, AWS EC2

- Deployed 4-service microservices platform on AWS EC2: Docker Compose orchestration with health checks, JWT-secured API gateway, Redis caching, PostgreSQL optimization, 15+ endpoint automated test suite.

Production REST API — Deployed & Monitored — [\[Repo\]](#) — [\[Live Demo\]](#)

Stack: Node.js/TypeScript, Docker, SQLite, Jest/Supertest, Render

- Containerized, deployed, and actively operates a live REST API: Dockerfile + Docker Compose, health check endpoint, rate limiting, API key auth, LLM response caching (~60% **AI cost reduction**), persistent volume management.